Week 1(Create, Alter, Drop and Insert)

Aim :

Write a query to create a new table **Emp** having the columns(attributes) ENo, EName, Job

Input Format :

+---------+---------+------+

| ENo | EName | Job |

+---------+---------+------+

| INT | VARCHAR | VARCHAR |

+---------+---------+------+

Output Format :

Algorithm :

Source Code :

```
CREATE TABLE Emp(ENo INT, EName VARCHAR(50), Job VARCHAR(50));
```

Aim :

Table: Emply

Write a SQL query to rename the column **EName** to **EmployeeName** in the **Emply** table. After renaming, use a **SELECT** statement to view the updated table.

+-------------+----------+

| Column Name | Type |

+-------------+----------+

| EMPN | int |

| ENAME | varchar |
| JOB | varchar |
+--------------+-----------+

Input Format :

+------------------+
| ENo | EName | Job |
+------------------+
+ 101 | John Doe | Software Engineer |
+ 102 | Jane Smith | Data Scientist |
+ 103 | Alice Johnson | Product Manager |
+------------------+

Output Format :

+----------------------------+
| ENo | EmployeeName | Job |
+----------------------------+
+ 101 | John Doe | Software Engineer |
+ 102 | Jane Smith | Data Scientist |
+ 103 | Alice Johnson | Product Manager |
+----------------------------+

Algorithm :

Source Code :

```
ALTER TABLE EMPLOYEE
DROP COLUMN HIREDATE;
SELECT * FROM EMPLOYEE;
```

Aim :

Table: employee

Write an SQL query to remove the **HIREDATE** column from the employee table. After dropping the column, use a **SELECT** statement to display the updated employee table and confirm that the **HIREDATE** column has been removed.

```
+-------------+-----------+
| Column Name | Type      |
+-------------+-----------+
| EMPN        | int       |
| ENAME       | varchar   |
| JOB         | varchar   |
| HIREDATE    | date      |
| SALARY      | float     |
| DEPTNO      | int       |
+-------------+-----------+
```

Each row of this table contains the EMPN, ENAME, JOB, HIREDATE, SALAR, and DEPTNO of an employee.

Input Format :

```
+------+--------+-----------+------------+-------+-------+
| EMPN | ENAME  | JOB       | HIREDATE   | SALARY| DEPTNO|
+------+--------+-----------+------------+-------+-------+
| 7369 | SMITH  | CLERK     | 17-DEC-80  | 800   | 20    |
| 7499 | ALLEN  | SALESMAN  | 20-FEB-81  | 1600  | 30    |
| 7521 | WARD   | SALESMAN  | 22-FEB-81  | 1250  | 30    |
| 7566 | JONES  | MANAGER   | 02-APR-81  | 2975  | 20    |
| 7654 | MARTIN | SALESMAN  | 28-SEP-81  | 1250  | 30    |
| 7698 | BLAKE  | MANAGER   | 01-MAY-81  | 2850  | 30    |
| 7782 | CLARK  | MANAGER   | 09-JUN-81  | 2450  | 10    |
| 7788 | SCOTT  | ANALYST   | 19-APR-87  | 3000  | 20    |
| 7839 | KING   | PRESIDENT | 17-NOV-81  | 5000  | 10    |
| 7844 | TURNER | SALESMAN  | 08-SEP-81  | 1500  | 30    |
| 7876 | ADAMS  | CLERK     | 23-MAY-87  | 1100  | 20    |
| 7900 | JAMES  | CLERK     | 03-DEC-81  | 950   | 30    |
| 7902 | FORD   | ANALYST   | 03-DEC-81  | 3000  | 20    |
| 7934 | MILLER | CLERK     | 23-JAN-82  | 1300  | 10    |
+------+--------+-----------+------------+-------+-------+
```

Output Format :

```
+------+--------+-----------+-------+-------+
| EMPN | ENAME  | JOB       | SALARY| DEPTNO|
+------+--------+-----------+-------+-------+
```

7369	SMITH	CLERK	800	20
7499	ALLEN	SALESMAN	1600	30
7521	WARD	SALESMAN	1250	30
7566	JONES	MANAGER	2975	20
7654	MARTIN	SALESMAN	1250	30
7698	BLAKE	MANAGER	2850	30
7782	CLARK	MANAGER	2450	10
7788	SCOTT	ANALYST	3000	20
7839	KING	PRESIDENT	5000	10
7844	TURNER	SALESMAN	1500	30
7876	ADAMS	CLERK	1100	20
7900	JAMES	CLERK	950	30
7902	FORD	ANALYST	3000	20
7934	MILLER	CLERK	1300	10
+------+--------+-----------+-------+-------+

Algorithm :

Source Code :

```
ALTER TABLE EMPLOYEE
DROP COLUMN HIREDATE;
SELECT * FROM EMPLOYEE;
```

Aim :

Table: Emp

Write a SQL query to insert multiple rows into the **Emp** table using a multi **INSERT** statement. After inserting, use a **SELECT** statement to view the updated table.

Input Format :

+---------+---------+------+
| ENo | EName | Job |
+---------+---------+------+

| INT | VARCHAR | VARCHAR |

+---------+---------+------+

Output Format :

+-----+----------+------------------+

| ENo | EName | Job |

+-----+----------+------------------+

| 101 | John Doe | Software Engineer |

| 102 | Jane Smith| Data Scientist |

| 103 | Alice Johnson | Product Manager |

+-----+----------+------------------+

Algorithm :

Source Code :

```
INSERT INTO Emp(ENo, EName, Job) VALUES(101, 'John Doe', 'Software Engineer');
INSERT INTO Emp(ENo, EName, Job) VALUES(102, 'Jane Smith', 'Data Scientist');
INSERT INTO Emp(ENo, EName, Job) VALUES(103, 'Alice Johnson', 'Product Manager');
SELECT * FROM Emp;
```

Aim :

Table: Employees

Write a SQL query to **ADD** an **AGE** column to the **Employees** table. After adding the column, update it with sample values and use a **SELECT** statement to view the updated table.

Input Format :

+---------+---------+------+

| ENo | EName | Job |

+---------+---------+------+

| INT | VARCHAR | VARCHAR |

+---------+---------+------+

Output Format :

```
+-----+--------------+------------------+-----+
| ENo | EName     | Job       | Age |
+-----+--------------+------------------+-----+
| 101 | John Doe   | Software Engineer | 28 |
| 102 | Jane Smith  | Data Scientist  | 32 |
| 103 | Alice Johnson | Product Manager  | 35 |
+-----+--------------+------------------+-----+
```

Algorithm :

Source Code :

```
ALTER TABLE Employees
ADD Age INT;

UPDATE Employees
SET Age = CASE
   WHEN ENo = 101 THEN 28
   WHEN ENo = 102 THEN 32
   WHEN ENo = 103 THEN 35
   ELSE NULL
END;
SELECT * FROM Employees;
```

Aim :

Write a Query to Delete the Rows?

You are given an employees table containing the following columns: EMPN (Employee Number), ENAME (Employee Name), JOB (Job Title), HIREDATE (Hire Date), SALARY (Salary), and DEPTNO (Department Number). The table contains records for several employees.

Your task is to delete the records for employees with the following EMPN values: 7900, 7839, and 7876. After executing the deletion, **use a SELECT statement** to view the updated table.

```
+--------------+-----------+
```

| Column Name | Type |
+-------------+----------+
EMPN	int
ENAME	varchar
JOB	varchar
HIREDATE	date
SALARY	float
DEPTNO	int
+-------------+----------+

Each row of this table contains the EMPN, ENAME, JOB, HIREDATE, SALAR, and DEPTNO of an employee.

Input Format :

Table: Employee

+------+--------+----------+-----------+-------+-------+
| EMPN | ENAME | JOB | HIREDATE | SALARY | DEPTNO|
+------+--------+----------+-----------+-------+-------+
7369	SMITH	CLERK	17-DEC-80	800	20
7499	ALLEN	SALESMAN	20-FEB-81	1600	30
7521	WARD	SALESMAN	22-FEB-81	1250	30
7566	JONES	MANAGER	02-APR-81	2975	20
7654	MARTIN	SALESMAN	28-SEP-81	1250	30
7698	BLAKE	MANAGER	01-MAY-81	2850	30
7782	CLARK	MANAGER	09-JUN-81	2450	10
7788	SCOTT	ANALYST	19-APR-87	3000	20
7839	KING	PRESIDENT	17-NOV-81	5000	10
7844	TURNER	SALESMAN	08-SEP-81	1500	30
7876	ADAMS	CLERK	23-MAY-87	1100	20
7900	JAMES	CLERK	03-DEC-81	950	30
7902	FORD	ANALYST	03-DEC-81	3000	20
7934	MILLER	CLERK	23-JAN-82	1300	10
+------+--------+----------+-----------+-------+-------+

Output Format :

+------+--------+----------+-----------+-------+-------+
| EMPN | ENAME | JOB | HIREDATE | SALARY| DEPTNO|
+------+--------+----------+-----------+-------+-------+
7369	SMITH	CLERK	1980-12-17	800	20
7499	ALLEN	SALESMAN	1981-02-20	1600	30
7521	WARD	SALESMAN	1981-02-22	1250	30

7566	JONES	MANAGER	1981-04-02	2975	20
7654	MARTIN	SALESMAN	1981-09-28	1250	30
7698	BLAKE	MANAGER	1981-05-01	2850	30
7782	CLARK	MANAGER	1981-06-09	2450	10
7788	SCOTT	ANALYST	1987-04-19	3000	20
7844	TURNER	SALESMAN	1981-09-08	1500	30
7902	FORD	ANALYST	1981-12-03	3000	20
7934	MILLER	CLERK	1982-01-23	1300	10
+------+--------+-----------+------------+-------+-------+

Algorithm :

Source Code :

```
DELETE FROM employees
WHERE EMPN IN (7900, 7839, 7876);

SELECT * FROM employees;
```

Week 2

Aim :

Table: Sailors

Write a SQL query to **find the sid of sailors who have a higher rating than "Horatio"**.

- If Horatio exists in the table, return only sailors with a **higher rating** than him.
- If Horatio **does not exist**, return **all sailors**.

+-------------+-----------+

| Column Name | Type |

+-------------+-----------+

| sid | int |

| sname | varchar |

| rating | int |

| age | float |

+-------------+-----------+

- sid is the primary key (unique identifier for each sailor).
- Each row of this table contains the sid, sname, rating, and age of a sailor.

Input Format :

Sailors table:

+-----+---------+--------+------+

| sid | sname | rating | age |

+-----+---------+--------+------+

| 22 | Dustin | 7 | 45.0 |

| 29 | Brutus | 1 | 33.0 |

| 31 | Lubber | 8 | 55.5 |

| 32 | Andy | 8 | 25.5 |

| 58 | Rusty | 10 | 35.0 |

| 71 | Zorba | 10 | 16.0 |

| 74 | Horatio | 9 | 35.0 |

| 85 | Art | 3 | 25.5 |

| 95 | Bob | 3 | 63.5 |

+-----+---------+--------+------+

Output Format :

+-----+

| sid |

+-----+

| 71 |
| 58 |
+-----+

Algorithm :

Source Code :

```
SELECT sid
FROM Sailors S
WHERE NOT EXISTS (
   SELECT 1
   FROM Sailors
   WHERE sname = 'Horatio' AND rating >= S.rating );
```

Aim :

Write a query for sailors with highest rating?

Table: Sailors

+--------------+-----------+
| Column Name | Type |
+--------------+-----------+
sid	int
sname	varchar
rating	int
age	float
+--------------+-----------+

- sid is the primary key (unique identifier for each sailor).
- Each row of this table contains the sid, sname, rating, and age of a sailor.

Input Format :

Sailors table:

+-----+---------+--------+------+

| sid | sname | rating | age |
+-----+---------+--------+------+
22	Dustin	7	45.0
29	Brutus	1	33.0
31	Lubber	8	55.5
32	Andy	8	25.5
58	Rusty	10	35.0
64	Horatio	7	35.0
71	Zorba	10	16.0
74	Horatio	9	35.0
85	Art	3	25.5
95	Bob	3	63.5
+-----+---------+--------+------+

Output Format :

+-----+--------+--------+
| sid | sname | rating |
+-----+--------+--------+
| 58 | Rusty | 10 |
| 71 | Zorba | 10 |
+-----+--------+--------+

Algorithm :

Source Code :

```
SELECT sid, sname, rating
FROM Sailors
WHERE rating = (SELECT MAX(rating) FROM Sailors);
```

Aim :

Task:

Write a SQL query to find the **names of sailors who have reserved at least one red boat and one green boat.**

Table: Sailors

+-------------+-----------+

| Column Name | Type |

+-------------+-----------+

| sid | int |

| sname | varchar |

| rating | int |

| age | float |

+-------------+-----------+

- sid is the primary key (unique identifier for each sailor).
- Each row of this table contains the sid, sname, rating, and age of a sailor.

Input Format :

Sailors table:

+-----+---------+--------+------+

| sid | sname | rating | age |

+-----+---------+--------+------+

| 22 | Dustin | 7 | 45.0 |

| 29 | Brutus | 1 | 33.0 |

| 31 | Lubber | 8 | 55.5 |

| 32 | Andy | 8 | 25.5 |

| 58 | Rusty | 10 | 35.0 |

| 64 | Horatio | 7 | 35.0 |

| 71 | Zorba | 10 | 16.0 |

| 74 | Horatio | 9 | 35.0 |

| 85 | Art | 3 | 25.5 |

| 95 | Bob | 3 | 63.5 |

+-----+---------+--------+------+

Reserves table:

+-----+-----+------------+

| sid | bid | Day |

+-----+-----+------------+

| 22 | 101 | 10/10/98 |

| 22 | 102 | 10/10/98 |

| 22 | 103 | 10/8/98 |

| 22 | 104 | 10/7/98 |

| 31 | 102 | 11/10/98 |

| 31 | 103 | 11/6/98 |

31	104	11/12/98
64	101	9/5/98
64	102	9/8/98
74	103	9/8/98
+-----+-----+------------+

Boats table:

+-----+-----------+--------+
| bid | bname | colour |
+-----+-----------+--------+
101	Interlake	blue
102	Interlake	red
103	Clipper	green
104	Marine	red
+-----+-----------+--------+

Output Format :

+--------+
| sname |
+--------+
| Dustin |
| Lubber |
+--------+

Algorithm :

Source Code :

```
SELECT sname
FROM Sailors
WHERE sid IN (
   SELECT sid FROM Reserves r
   JOIN Boats b ON r.bid = b.bid
   WHERE b.colour = 'red'
)
AND sid IN (
   SELECT sid FROM Reserves r
   JOIN Boats b ON r.bid = b.bid
   WHERE b.colour = 'green'
);
```

Aim :

Task

Write a query for the names of sailors who have reserve red boat but not green boat?

+--------------+-----------+

| Column Name | Type |

+--------------+-----------+

| sid | int |

| sname | varchar |

| rating | int |

| age | float |

+--------------+-----------+

- sid is the primary key (unique identifier for each sailor).
- Each row of this table contains the sid, sname, rating, and age of a sailor.

Input Format :

Sailors table:

+-----+---------+--------+------+

| sid | sname | rating | age |

+-----+---------+--------+------+

| 22 | Dustin | 7 | 45.0 |

| 29 | Brutus | 1 | 33.0 |

| 31 | Lubber | 8 | 55.5 |

| 32 | Andy | 8 | 25.5 |

| 58 | Rusty | 10 | 35.0 |

64	Horatio	7	35.0
71	Zorba	10	16.0
74	Horatio	9	35.0
85	Art	3	25.5
95	Bob	3	63.5
+-----+---------+--------+------+

Reserves table:

+-----+-----+------------+
| sid | bid | Day |
+-----+-----+------------+
22	101	10/10/98
22	102	10/10/98
22	103	10/8/98
22	104	10/7/98
31	102	11/10/98
31	103	11/6/98
31	104	11/12/98
64	101	9/5/98
64	102	9/8/98
74	103	9/8/98
+-----+-----+------------+

Boats table:

+-----+-----------+--------+
| bid | bname | colour |
+-----+-----------+--------+
101	Interlake	blue
102	Interlake	red
103	Clipper	green
104	Marine	red
+-----+-----------+--------+

Output Format :

+--------+
| sname |
+--------+
| Horatio|
+--------+

Algorithm :

Source Code :

```
SELECT s.sname
FROM Sailors s
JOIN Reserves r ON s.sid = r.sid
JOIN Boats b ON r.bid = b.bid
WHERE b.colour = 'red'
AND s.sid NOT IN (
   SELECT s.sid
   FROM Sailors s
   JOIN Reserves r ON s.sid = r.sid
   JOIN Boats b ON r.bid = b.bid
   WHERE b.colour = 'green'
)
```

Aim :

Task

Write a SQL query to find the names of sailors who have **reserved at least one red boat and at least one blue boat**.

Table

+--------------+-----------+

| Column Name | Type |

+--------------+-----------+

| sid | int |

| sname | varchar |

| rating | int |

| age | float |

+--------------+-----------+

- sid is the primary key (unique identifier for each sailor).
- Each row of this table contains the sid, sname, rating, and age of a sailor.

Input Format :

Sailors table:

+-----+---------+--------+------+

| sid | sname | rating | age |

+-----+---------+--------+------+

| 22 | Dustin | 7 | 45.0 |

| 29 | Brutus | 1 | 33.0 |

| 31 | Lubber | 8 | 55.5 |

| 32 | Andy | 8 | 25.5 |

| 58 | Rusty | 10 | 35.0 |

| 64 | Horatio | 7 | 35.0 |

| 71 | Zorba | 10 | 16.0 |

| 74 | Horatio | 9 | 35.0 |

| 85 | Art | 3 | 25.5 |

| 95 | Bob | 3 | 63.5 |

+-----+---------+--------+------+

Reserves table:

+-----+-----+------------+

| sid | bid | Day |

+-----+-----+------------+

| 22 | 101 | 10/10/98 |

| 22 | 102 | 10/10/98 |

| 22 | 103 | 10/8/98 |

| 22 | 104 | 10/7/98 |

| 31 | 102 | 11/10/98 |

| 31 | 103 | 11/6/98 |

| 31 | 104 | 11/12/98 |

| 64 | 101 | 9/5/98 |

| 64 | 102 | 9/8/98 |

| 74 | 103 | 9/8/98 |

+-----+-----+------------+

Boats table:

+-----+-----------+--------+

| bid | bname | colour |

+-----+-----------+--------+

| 101 | Interlake | blue |

| 102 | Interlake | red |

| 103 | Clipper | green |

| 104 | Marine | red |

+-----+-----------+--------+

Output Format :

+--------+
| sname |
+--------+
| Dustin |
| Horatio|
+--------+

Algorithm :

Source Code :

```
SELECT s.sname
FROM Sailors s
WHERE s.sid IN (
    SELECT r.sid FROM Reserves r
    JOIN Boats b ON r.bid = b.bid
    WHERE b.colour = 'red'
    INTERSECT
    SELECT r.sid FROM Reserves r
    JOIN Boats b ON r.bid = b.bid
    WHERE b.colour = 'blue'
);
```

Aim :

Task

Write a SQL query to find the names of sailors who **have NOT reserved both a blue and a green boat** using **NOT EXISTS**.

Table

+-------------+-----------+
| Column Name | Type |

+--------------+-----------+

| sid | int |

| sname | varchar |

| rating | int |

| age | float |

+--------------+-----------+

- sid is the primary key (unique identifier for each sailor).
- Each row of this table contains the sid, sname, rating, and age of a sailor.

Input Format :

Sailors table:

+-----+---------+--------+------+

| sid | sname | rating | age |

+-----+---------+--------+------+

| 22 | Dustin | 7 | 45.0 |

| 29 | Brutus | 1 | 33.0 |

| 31 | Lubber | 8 | 55.5 |

| 32 | Andy | 8 | 25.5 |

| 58 | Rusty | 10 | 35.0 |

| 64 | Horatio | 7 | 35.0 |

| 71 | Zorba | 10 | 16.0 |

| 74 | Horatio | 9 | 35.0 |

| 85 | Art | 3 | 25.5 |

| 95 | Bob | 3 | 63.5 |

+-----+---------+--------+------+

Reserves table:

+-----+-----+------------+

| sid | bid | Day |

+-----+-----+------------+

| 22 | 101 | 10/10/98 |

| 22 | 102 | 10/10/98 |

| 22 | 103 | 10/8/98 |

| 22 | 104 | 10/7/98 |

| 31 | 102 | 11/10/98 |

| 31 | 103 | 11/6/98 |

| 31 | 104 | 11/12/98 |

| 64 | 101 | 9/5/98 |

| 64 | 102 | 9/8/98 |

| 74 | 103 | 9/8/98 |

+-----+-----+------------+

Boats table:

+-----+----------+--------+

| bid | bname | colour |

+-----+----------+--------+

| 101 | Interlake | blue |

| 102 | Interlake | red |

| 103 | Clipper | green |

| 104 | Marine | red |

+-----+----------+--------+

Output Format :

+--------+

| sname |

+--------+

| Brutus |

| Lubber |

| Andy |

| Rusty |

| Horatio|

| Zorba |

| Horatio|

| Art |

| Bob |

+--------+

Algorithm :

Source Code :

```
SELECT s.sname
FROM Sailors s
WHERE NOT EXISTS (
  SELECT 1
  FROM Reserves r
  JOIN Boats b ON r.bid = b.bid
  WHERE s.sid = r.sid AND b.colour = 'blue'
)
OR NOT EXISTS (
  SELECT 1
  FROM Reserves r
  JOIN Boats b ON r.bid = b.bid
  WHERE s.sid = r.sid AND b.colour = 'green'
);
```

Aim :

Task

Write a SQL query to find the names of sailors who **have reserved a boat with ID 103.**

Table

+--------------+-----------+

| Column Name | Type |

+--------------+-----------+

| sid | int |

| sname | varchar |

| rating | int |

| age | float |

+--------------+-----------+

- sid is the primary key (unique identifier for each sailor).
- Each row of this table contains the sid, sname, rating, and age of a sailor.

Input Format :

Sailors table:

+-----+---------+--------+------+

| sid | sname | rating | age |

+-----+---------+--------+------+

| 22 | Dustin | 7 | 45.0 |

| 29 | Brutus | 1 | 33.0 |

31	Lubber	8	55.5
32	Andy	8	25.5
58	Rusty	10	35.0
64	Horatio	7	35.0
71	Zorba	10	16.0
74	Horatio	9	35.0
85	Art	3	25.5
95	Bob	3	63.5
+-----+---------+--------+------+

Reserves table:

+-----+-----+------------+
| sid | bid | Day |
+-----+-----+------------+
22	101	10/10/98
22	102	10/10/98
22	103	10/8/98
22	104	10/7/98
31	102	11/10/98
31	103	11/6/98
31	104	11/12/98
64	101	9/5/98
64	102	9/8/98
74	103	9/8/98
+-----+-----+------------+

Boats table:

+-----+-----------+--------+
| bid | bname | colour |
+-----+-----------+--------+
101	Interlake	blue
102	Interlake	red
103	Clipper	green
104	Marine	red
+-----+-----------+--------+

Output Format :

+--------+
| sname |
+--------+
| Dustin |
| Lubber |

| Horatio|

+--------+

Algorithm :

Source Code :

```
SELECT s.sname
FROM Sailors s
JOIN Reserves r ON s.sid = r.sid
WHERE r.bid = 103;
```

Week - 3

Aim :

Write a query to display the no of different employee name from employee

Table: Employee

+--------------+-----------+
| Column Name | Type |
+--------------+-----------+
EMPN	int
ENAME	varchar
JOB	varchar
HIREDATE	date
SALARY	float
DEPTNO	int
+--------------+-----------+

Each row of this table contains the EMPN, ENAME, JOB, HIREDATE, SALAR, and DEPTNO of an employee.

Input Format :

+------+--------+-----------+------------+-------+-------+
| EMPN | ENAME | JOB | HIREDATE | SALARY | DEPTNO|
+------+--------+-----------+------------+-------+-------+
7369	SMITH	CLERK	17-DEC-80	800	20
7499	ALLEN	SALESMAN	20-FEB-81	1600	30
7521	WARD	SALESMAN	22-FEB-81	1250	30
7566	JONES	MANAGER	02-APR-81	2975	20
7654	MARTIN	SALESMAN	28-SEP-81	1250	30
7698	BLAKE	MANAGER	01-MAY-81	2850	30
7782	CLARK	MANAGER	09-JUN-81	2450	10
7788	SCOTT	ANALYST	19-APR-87	3000	20
7839	KING	PRESIDENT	17-NOV-81	5000	10
7844	TURNER	SALESMAN	08-SEP-81	1500	30
7876	ADAMS	CLERK	23-MAY-87	1100	20
7900	JAMES	CLERK	03-DEC-81	950	30
7902	FORD	ANALYST	03-DEC-81	3000	20
7934	MILLER	CLERK	23-JAN-82	1300	10
+------+--------+-----------+------------+-------+-------+

Output Format :

14

Algorithm :

Source Code :

```
SELECT COUNT(DISTINCT ENAME) AS unique_employee_names
FROM Employee;
```

Aim :

Write a query to add sum of all employees salaries

Table: Employee

+--------------+-----------+
| Column Name | Type |
+--------------+-----------+
EMPN	int
ENAME	varchar
JOB	varchar
HIREDATE	date
SALARY	float
DEPTNO	int
+--------------+-----------+

Each row of this table contains the EMPN, ENAME, JOB, HIREDATE, SALAR, and DEPTNO of an employee.

Input Format :

+------+--------+-----------+------------+-------+-------+
| EMPN | ENAME | JOB | HIREDATE | SALARY | DEPTNO|

+------+--------+----------+------------+-------+-------+
7369	SMITH	CLERK	17-DEC-80	800	20
7499	ALLEN	SALESMAN	20-FEB-81	1600	30
7521	WARD	SALESMAN	22-FEB-81	1250	30
7566	JONES	MANAGER	02-APR-81	2975	20
7654	MARTIN	SALESMAN	28-SEP-81	1250	30
7698	BLAKE	MANAGER	01-MAY-81	2850	30
7782	CLARK	MANAGER	09-JUN-81	2450	10
7788	SCOTT	ANALYST	19-APR-87	3000	20
7839	KING	PRESIDENT	17-NOV-81	5000	10
7844	TURNER	SALESMAN	08-SEP-81	1500	30
7876	ADAMS	CLERK	23-MAY-87	1100	20
7900	JAMES	CLERK	03-DEC-81	950	30
7902	FORD	ANALYST	03-DEC-81	3000	20
7934	MILLER	CLERK	23-JAN-82	1300	10
+------+--------+----------+------------+-------+-------+

Output Format :

29025.0

Algorithm :

Source Code :

```
SELECT SUM(SALARY) AS total_salary
FROM Employee;
```

Aim :

Write a query to add the different salaries of all employees

Table: Employee.

+-------------+----------+
| Column Name | Type |

+--------------+-----------+
EMPN	int
ENAME	varchar
JOB	varchar
HIREDATE	date
SALARY	float
DEPTNO	int
+--------------+-----------+

Each row of this table contains the EMPN, ENAME, JOB, HIREDATE, SALAR, and DEPTNO of an employee.

Input Format :

+------+--------+-----------+------------+-------+-------+
| EMPN | ENAME | JOB | HIREDATE | SALARY | DEPTNO|
+------+--------+-----------+------------+-------+-------+
7369	SMITH	CLERK	17-DEC-80	800	20
7499	ALLEN	SALESMAN	20-FEB-81	1600	30
7521	WARD	SALESMAN	22-FEB-81	1250	30
7566	JONES	MANAGER	02-APR-81	2975	20
7654	MARTIN	SALESMAN	28-SEP-81	1250	30
7698	BLAKE	MANAGER	01-MAY-81	2850	30
7782	CLARK	MANAGER	09-JUN-81	2450	10
7788	SCOTT	ANALYST	19-APR-87	3000	20
7839	KING	PRESIDENT	17-NOV-81	5000	10
7844	TURNER	SALESMAN	08-SEP-81	1500	30
7876	ADAMS	CLERK	23-MAY-87	1100	20
7900	JAMES	CLERK	03-DEC-81	950	30
7902	FORD	ANALYST	03-DEC-81	3000	20
7934	MILLER	CLERK	23-JAN-82	1300	10
+------+--------+-----------+------------+-------+-------+

Output Format :

24775.0

Algorithm :

Source Code :

```
SELECT SUM(DISTINCT SALARY) AS total_distinct_salaries
FROM Employee;
```

Aim :

Write a query to find the avg salary of all the employees

+--------------+-----------+

| Column Name | Type |

+--------------+-----------+

| EMPN | int |

| ENAME | varchar |

| JOB | varchar |

| HIREDATE | date |

| SALARY | float |

| DEPTNO | int |

+--------------+-----------+

Each row of this table contains the EMPN, ENAME, JOB, HIREDATE, SALAR, and DEPTNO of an employee.

Input Format :

+------+--------+-----------+------------+-------+-------+

| EMPN | ENAME | JOB | HIREDATE | SALARY | DEPTNO|

+------+--------+-----------+------------+-------+-------+

| 7369 | SMITH | CLERK | 17-DEC-80 | 800 | 20 |

| 7499 | ALLEN | SALESMAN | 20-FEB-81 | 1600 | 30 |

| 7521 | WARD | SALESMAN | 22-FEB-81 | 1250 | 30 |

| 7566 | JONES | MANAGER | 02-APR-81 | 2975 | 20 |

| 7654 | MARTIN | SALESMAN | 28-SEP-81 | 1250 | 30 |

| 7698 | BLAKE | MANAGER | 01-MAY-81 | 2850 | 30 |

| 7782 | CLARK | MANAGER | 09-JUN-81 | 2450 | 10 |

| 7788 | SCOTT | ANALYST | 19-APR-87 | 3000 | 20 |

| 7839 | KING | PRESIDENT | 17-NOV-81 | 5000 | 10 |

| 7844 | TURNER | SALESMAN | 08-SEP-81 | 1500 | 30 |

| 7876 | ADAMS | CLERK | 23-MAY-87 | 1100 | 20 |

| 7900 | JAMES | CLERK | 03-DEC-81 | 950 | 30 |

| 7902 | FORD | ANALYST | 03-DEC-81 | 3000 | 20 |

| 7934 | MILLER | CLERK | 23-JAN-82 | 1300 | 10 |

+------+--------+----------+------------+-------+-------+

Output Format :

2073.21428571429

Algorithm :

Source Code :

```
SELECT AVG(SALARY) AS average_salary
FROM Employee;
```

Aim :

Write a query to find the highest salary from all employees

+--------------+-----------+
| Column Name | Type |
+--------------+-----------+
EMPN	int
ENAME	varchar
JOB	varchar
HIREDATE	date
SALARY	float
DEPTNO	int
+--------------+-----------+

Each row of this table contains the EMPN, ENAME, JOB, HIREDATE, SALAR, and DEPTNO of an employee.

Input Format :

+------+--------+----------+------------+-------+-------+
| EMPN | ENAME | JOB | HIREDATE | SALARY | DEPTNO|
+------+--------+----------+------------+-------+-------+

7369	SMITH	CLERK	17-DEC-80	800	20
7499	ALLEN	SALESMAN	20-FEB-81	1600	30
7521	WARD	SALESMAN	22-FEB-81	1250	30
7566	JONES	MANAGER	02-APR-81	2975	20
7654	MARTIN	SALESMAN	28-SEP-81	1250	30
7698	BLAKE	MANAGER	01-MAY-81	2850	30
7782	CLARK	MANAGER	09-JUN-81	2450	10
7788	SCOTT	ANALYST	19-APR-87	3000	20
7839	KING	PRESIDENT	17-NOV-81	5000	10
7844	TURNER	SALESMAN	08-SEP-81	1500	30
7876	ADAMS	CLERK	23-MAY-87	1100	20
7900	JAMES	CLERK	03-DEC-81	950	30
7902	FORD	ANALYST	03-DEC-81	3000	20
7934	MILLER	CLERK	23-JAN-82	1300	10
+------+--------+-----------+------------+-------+-------+

Output Format :

5000.0

Algorithm :

Source Code :

```
SELECT MAX(SALARY) AS highest_salary
FROM Employee;
```

Aim :

Write a query to find the lowest salary from all employees

+--------------+-----------+
| Column Name | Type |
+--------------+-----------+
| EMPN | int |

ENAME	varchar
JOB	varchar
HIREDATE	date
SALARY	float
DEPTNO	int
+--------------+-----------+

Each row of this table contains the EMPN, ENAME, JOB, HIREDATE, SALAR, and DEPTNO of an employee.

Input Format :

+------+--------+-----------+------------+-------+-------+
| EMPN | ENAME | JOB | HIREDATE | SALARY | DEPTNO|
+------+--------+-----------+------------+-------+-------+
7369	SMITH	CLERK	17-DEC-80	800	20
7499	ALLEN	SALESMAN	20-FEB-81	1600	30
7521	WARD	SALESMAN	22-FEB-81	1250	30
7566	JONES	MANAGER	02-APR-81	2975	20
7654	MARTIN	SALESMAN	28-SEP-81	1250	30
7698	BLAKE	MANAGER	01-MAY-81	2850	30
7782	CLARK	MANAGER	09-JUN-81	2450	10
7788	SCOTT	ANALYST	19-APR-87	3000	20
7839	KING	PRESIDENT	17-NOV-81	5000	10
7844	TURNER	SALESMAN	08-SEP-81	1500	30
7876	ADAMS	CLERK	23-MAY-87	1100	20
7900	JAMES	CLERK	03-DEC-81	950	30
7902	FORD	ANALYST	03-DEC-81	3000	20
7934	MILLER	CLERK	23-JAN-82	1300	10
+------+--------+-----------+------------+-------+-------+

Output Format :

800.0

Algorithm :

Source Code :

```
SELECT MIN(SALARY) AS lowest_salary
FROM Employee;
```

Aim :

Write a query for age of youngest sailor for each rating level?

Table: Sailors

+--------------+-----------+

| Column Name | Type |

+--------------+-----------+

| sid | int |

| sname | varchar |

| rating | int |

| age | float |

+--------------+-----------+

- sid is the primary key (unique identifier for each sailor).
- Each row of this table contains the sid, sname, rating, and age of a sailor.

Input Format :

Sailors table:

+-----+---------+--------+------+

| sid | sname | rating | age |

+-----+---------+--------+------+

| 22 | Dustin | 7 | 45.0 |

| 29 | Brutus | 1 | 33.0 |

| 31 | Lubber | 8 | 55.5 |

| 32 | Andy | 8 | 25.5 |

| 58 | Rusty | 10 | 35.0 |

| 64 | Horatio | 7 | 35.0 |

| 71 | Zorba | 10 | 16.0 |

| 74 | Horatio | 9 | 35.0 |

| 85 | Art | 3 | 25.5 |

| 95 | Bob | 3 | 63.5 |

+-----+---------+--------+------+

Output Format :

1|33.0
3|25.5
7|35.0
8|25.5
9|35.0
10|16.0

Algorithm :

Source Code :

```
SELECT rating, MIN(age) AS youngest_age
FROM Sailors
GROUP BY rating;
```

Aim :

Write a query to find the age of youngest sailors who is elizible to vote for each rating level with atleast two such sailors?

Table: Sailors

+--------------+-----------+
| Column Name | Type |
+--------------+-----------+
sid	int
sname	varchar
rating	int
age	float
+--------------+-----------+

- sid is the primary key (unique identifier for each sailor).
- Each row of this table contains the sid, sname, rating, and age of a sailor.

Input Format :

Sailors table:

+-----+---------+--------+------+

| sid | sname | rating | age |

+-----+---------+--------+------+

| 22 | Dustin | 7 | 45.0 |

| 29 | Brutus | 1 | 33.0 |

| 31 | Lubber | 8 | 55.5 |

| 32 | Andy | 8 | 25.5 |

| 58 | Rusty | 10 | 35.0 |

| 64 | Horatio | 7 | 35.0 |

| 71 | Zorba | 10 | 16.0 |

| 74 | Horatio | 9 | 35.0 |

| 85 | Art | 3 | 25.5 |

| 95 | Bob | 3 | 63.5 |

+-----+---------+--------+------+

Output Format :

3|25.5

7|35.0

8|25.5

Algorithm :

Source Code :

```
SELECT rating, MIN(age) AS youngest_age
FROM Sailors
WHERE age >= 18
GROUP BY rating
HAVING COUNT(sid) >= 2;
```

Aim :

Write a query for each red boat finding the no of reservations for this boat?

Table

+--------------+-----------+

| Column Name | Type |

+--------------+-----------+

| sid | int |

| sname | varchar |

| rating | int |

| age | float |

+--------------+-----------+

- sid is the primary key (unique identifier for each sailor).
- Each row of this table contains the sid, sname, rating, and age of a sailor.

Input Format :

Sailors table:

+-----+---------+--------+------+

| sid | sname | rating | age |

+-----+---------+--------+------+

| 22 | Dustin | 7 | 45.0 |

| 29 | Brutus | 1 | 33.0 |

| 31 | Lubber | 8 | 55.5 |

| 32 | Andy | 8 | 25.5 |

| 58 | Rusty | 10 | 35.0 |

| 64 | Horatio | 7 | 35.0 |

| 71 | Zorba | 10 | 16.0 |

| 74 | Horatio | 9 | 35.0 |

| 85 | Art | 3 | 25.5 |

| 95 | Bob | 3 | 63.5 |

+-----+---------+--------+------+

Reserves table:

+-----+-----+------------+

| sid | bid | Day |

+-----+-----+------------+

| 22 | 101 | 10/10/98 |

| 22 | 102 | 10/10/98 |

| 22 | 103 | 10/8/98 |

| 22 | 104 | 10/7/98 |

| 31 | 102 | 11/10/98 |

| 31 | 103 | 11/6/98 |

| 31 | 104 | 11/12/98 |

| 64 | 101 | 9/5/98 |

| 64 | 102 | 9/8/98 |
| 74 | 103 | 9/8/98 |
+-----+-----+------------+

Boats table:

+-----+-----------+--------+
| bid | bname | colour |
+-----+-----------+--------+
101	Interlake	blue
102	Interlake	red
103	Clipper	green
104	Marine	red
+-----+-----------+--------+

Output Format :

+------------+-------------------+
| bname | reservation_count |
+------------+-------------------+
| Interlake | 3 |
| Marine | 2 |
+------------+-------------------+

Algorithm :

Source Code :

```
SELECT b.bname, COUNT(r.sid) AS reservation_count
FROM Reserves r
JOIN Boats b ON r.bid = b.bid
WHERE b.colour = 'red'
GROUP BY b.bname;
```

Aim :

Write a query to find avg age of sailors for each rating level that has atleast two sailors?

Table: Sailors

+-------------+----------+

| Column Name | Type |

+-------------+----------+

| sid | int |

| sname | varchar |

| rating | int |

| age | float |

+-------------+----------+

- sid is the primary key (unique identifier for each sailor).
- Each row of this table contains the sid, sname, rating, and age of a sailor.

Input Format :

Sailors table:

+-----+---------+--------+------+

| sid | sname | rating | age |

+-----+---------+--------+------+

| 22 | Dustin | 7 | 45.0 |

| 29 | Brutus | 1 | 33.0 |

| 31 | Lubber | 8 | 55.5 |

| 32 | Andy | 8 | 25.5 |

| 58 | Rusty | 10 | 35.0 |

| 64 | Horatio | 7 | 35.0 |

| 71 | Zorba | 10 | 16.0 |

| 74 | Horatio | 9 | 35.0 |

| 85 | Art | 3 | 25.5 |

| 95 | Bob | 3 | 63.5 |

+-----+---------+--------+------+

Output Format :

+--------+------+

| rating | age |

+--------+------+

| 3 | 44.5 |

| 7 | 40.0 |

| 8 | 40.5 |

| 10 | 25.5 |

+--------+------+

Algorithm :

Source Code :

```
SELECT rating, AVG(age) AS avg_age
FROM Sailors
GROUP BY rating
HAVING COUNT(sid) >= 2;
```

Aim :

Write a query avg age of sailors who are eligible to vote for each rating level that has atleast two sailors?

Table: Sailors

+--------------+-----------+

| Column Name | Type |

+--------------+-----------+

| sid | int |

| sname | varchar |

| rating | int |

| age | float |

+--------------+-----------+

- sid is the primary key (unique identifier for each sailor).
- Each row of this table contains the sid, sname, rating, and age of a sailor.

Input Format :

Sailors table:

+-----+---------+--------+------+

| sid | sname | rating | age |

+-----+---------+--------+------+

| 22 | Dustin | 7 | 45.0 |

| 29 | Brutus | 1 | 33.0 |

| 31 | Lubber | 8 | 55.5 |

| 32 | Andy | 8 | 25.5 |

| 58 | Rusty | 10 | 35.0 |

| 64 | Horatio | 7 | 35.0 |

71	Zorba	10	16.0
74	Horatio	9	35.0
85	Art	3	25.5
95	Bob	3	63.5
+-----+---------+--------+------+

Output Format :

+--------+------+
| rating | age |
+--------+------+
3	44.5
7	40.0
8	40.5
+--------+------+

Algorithm :

Source Code :

```
SELECT rating, AVG(age) AS avg_age
FROM Sailors
WHERE age >= 18
GROUP BY rating
HAVING COUNT(sid) >= 2;
```

Aim :

Table: employee

Write a query to display the total number of records in the employee table.

Requirements:

- Use the **COUNT** aggregate function to calculate the total number of records.

+--------------+-----------+
| Column Name | Type |
+--------------+-----------+
EMPN	int
ENAME	varchar
JOB	varchar
HIREDATE	date
SALARY	float
DEPTNO	int
+--------------+-----------+

Each row of this table contains the EMPN, ENAME, JOB, HIREDATE, SALAR, and DEPTNO of an employee.

Input Format :

+------+--------+-----------+------------+-------+-------+
| EMPN | ENAME | JOB | HIREDATE | SALARY | DEPTNO|
+------+--------+-----------+------------+-------+-------+
7369	SMITH	CLERK	17-DEC-80	800	20
7499	ALLEN	SALESMAN	20-FEB-81	1600	30
7521	WARD	SALESMAN	22-FEB-81	1250	30
7566	JONES	MANAGER	02-APR-81	2975	20
7654	MARTIN	SALESMAN	28-SEP-81	1250	30
7698	BLAKE	MANAGER	01-MAY-81	2850	30
7782	CLARK	MANAGER	09-JUN-81	2450	10
7788	SCOTT	ANALYST	19-APR-87	3000	20
7839	KING	PRESIDENT	17-NOV-81	5000	10
7844	TURNER	SALESMAN	08-SEP-81	1500	30
7876	ADAMS	CLERK	23-MAY-87	1100	20
7900	JAMES	CLERK	03-DEC-81	950	30
7902	FORD	ANALYST	03-DEC-81	3000	20
7934	MILLER	CLERK	23-JAN-82	1300	10
+------+--------+-----------+------------+-------+-------+

Output Format :

14

Algorithm :

Source Code :

```
SELECT COUNT(*) AS total_records
FROM employee;
```

Aim :

Table: empolyee

write a SQL query to count the total number of employee names stored in the **employee** table and display the result with the column name **total_employee_names**?

+--------------+-----------+

| Column Name | Type |

+--------------+-----------+

| EMPN | int |

| ENAME | varchar |

| JOB | varchar |

| HIREDATE | date |

| SALARY | float |

| DEPTNO | int |

+--------------+-----------+

Each row of this table contains the EMPN, ENAME, JOB, HIREDATE, SALAR, and DEPTNO of an employee.

Input Format :

+------+--------+-----------+------------+-------+-------+

| EMPN | ENAME | JOB | HIREDATE | SALARY | DEPTNO|

+------+--------+-----------+------------+-------+-------+

| 7369 | SMITH | CLERK | 17-DEC-80 | 800 | 20 |

| 7499 | ALLEN | SALESMAN | 20-FEB-81 | 1600 | 30 |

| 7521 | WARD | SALESMAN | 22-FEB-81 | 1250 | 30 |

| 7566 | JONES | MANAGER | 02-APR-81 | 2975 | 20 |

| 7654 | MARTIN | SALESMAN | 28-SEP-81 | 1250 | 30 |

| 7698 | BLAKE | MANAGER | 01-MAY-81 | 2850 | 30 |

| 7782 | CLARK | MANAGER | 09-JUN-81 | 2450 | 10 |

| 7788 | SCOTT | ANALYST | 19-APR-87 | 3000 | 20 |

| 7839 | KING | PRESIDENT | 17-NOV-81 | 5000 | 10 |

| 7844 | TURNER | SALESMAN | 08-SEP-81 | 1500 | 30 |

7876	ADAMS	CLERK	23-MAY-87	1100	20
7900	JAMES	CLERK	03-DEC-81	950	30
7902	FORD	ANALYST	03-DEC-81	3000	20
7934	MILLER	CLERK	23-JAN-82	1300	10
+------+--------+-----------+------------+-------+-------+

Output Format :

14

Algorithm :

Source Code :

```
select count(Ename) as table_employee_names from employee;
```

Week 4

Aim :

Write an SQL query to retrieve the **current year** from the system date. The query should return the year as a 4-digit number.

Input Format :

NO INPUT REQUIRED

Output Format :

2025

Algorithm :

Source Code :

```
SELECT strftime('%Y', CURRENT_DATE) AS formatted_year;
```

Aim :

Problem Statement: Extract the Current Month in SQLite

Write an SQL query to extract the **current month** from the system date in **SQLite**. The query should return the month in **numerical format** (e.g., 01 for January, 02 for February).

Input:

The system date (e.g., 2025-01-06).

Output:

The current month in **two-digit numerical format** (e.g., 01 for January, 02 for February).

Example:

If today's date is 2025-01-06, the output should be:

formatted_month01
This query should return the month as a **two-digit number** representing the current month.

Input Format :

Input Format:
The input is **implicit** and comes from the system's current date and time, which is automatically retrieved by SQLite using the strftime function.

- **Input Source**: NOW (current date and time).
- **Format of NOW**: YYYY-MM-DD HH:MM:SS (e.g., 2025-01-06 14:30:45).

You do not need to manually provide the date as it is automatically taken from the system's current date and time when the query is executed. The SQLite engine will handle it internally using the strftime function.

Output Format :

Output Format:
The output will be a **single column** containing the **current month** in **two-digit numerical format** (e.g., 01 for January, 02 for February).

- **Column Name**: formatted_month
- **Value Format**: A two-digit string representing the current month.

Example Output:
formatted_month01
If the current system date is 2025-01-06, the result will be 01 representing January.

Algorithm :

Source Code :

```
SELECT strftime('%m', 'now') AS formatted_month;
```

Aim :

Problem Statement:

Write an SQL query to retrieve the current day of the week using the TO_CHAR() function, formatted as the full name of the day (e.g., "MONDAY", "TUESDAY"). Ensure the day name is displayed in uppercase and padded with spaces to maintain consistent width. Use the SYSDATE function to fetch the current system date.

Input Format :

Input Format:

The query does not require any external input, as it uses the SYSDATE function to fetch the current system date and time. The input is implicitly the system's current date.

No parameters or user inputs are needed. The query is executed directly to return the desired result.

Output Format :

Output Format:

The query will return a single column with the current day of the week formatted as follows:

- The day name will be in uppercase (e.g., "MONDAY", "TUESDAY").
- The output will be padded with spaces to match the default width of the DAY format in Oracle, ensuring alignment in the results.

Algorithm :

Source Code :

```
SELECT strftime('%w', '2025-03-12') AS current_day_number, -- A Wednesday date
    CASE strftime('%w', '2025-03-12')
      WHEN '0' THEN 'SUNDAY'
      WHEN '1' THEN 'MONDAY'
      WHEN '2' THEN 'TUESDAY'
      WHEN '3' THEN 'WEDNESDAY'
      WHEN '4' THEN 'THURSDAY'
      WHEN '5' THEN 'FRIDAY'
      WHEN '6' THEN 'SATURDAY'
    END AS current_day_name;
```

Week 5(Commit, Rollback, Savepoint and Set Transaction Commands)

Aim :

Update the salaries of all employees in the "IT" department by 10%. Ensure the changes are made within a transaction, and commit the updates to the database. Verify that only the "IT" department employees' salaries are updated.

+--------------+-----------+
| Column Name | Type |
+--------------+-----------+
employee_id	int
first_name	varchar
last_name	varchar
department	varchar
salary	decimal
+--------------+-----------+

Input Format :

employee_id	first_name	last_name	department	salary
1	Alice	Smith	IT	60000
2	Bob	Johnson	HR	55000
3	Charlie	Lee	IT	65000
4	David	Williams	Finance	70000
5	Eva	Davis	IT	58000

Output Format :

1|Alice|Smith|IT|66000
2|Bob|Johnson|HR|55000
3|Charlie|Lee|IT|71500
4|David|Williams|Finance|70000
5|Eva|Davis|IT|63800.0

Algorithm :

Source Code :

```
BEGIN;

UPDATE employees
SET salary = salary * 1.10  -- Increase salary by 10%
WHERE department = 'IT';

COMMIT;
SELECT * FROM employees;
```

Aim :

Table: BankAccounts

You need to simulate a bank transaction(from the table BankAccounts) where $400 is deducted from Alice's account. If Alice's balance becomes negative, the transaction should be **rolled back**, and a message should indicate that the transaction was **unsuccessful**. If the transaction is successful, **commit** the changes and display a success message.

```
+-----------------+-----------+
| Column Name   | Type   |
+-----------------+-----------+
| AccountID    | int    |
| AccountHolder  | varchar  |
| Balance     | decimal  |
+-----------------+-----------+
```

Input Format :

```
+------------+-----------------+---------+
| AccountID | AccountHolder  | Balance |
+------------+-----------------+---------+
| 1    | Alice      | 300.00 |
+------------+-----------------+---------+
```

Output Format :

-100.0
Transaction rolled back! Insufficient funds.
1|Alice|300.0

(Transaction succesful!) Otherwise

Algorithm :

Source Code :

```
BEGIN TRANSACTION;

UPDATE BankAccounts
SET Balance = Balance - 400
WHERE AccountID = 1;

SELECT Balance FROM BankAccounts WHERE AccountID = 1;

SELECT CASE
   WHEN (SELECT Balance FROM BankAccounts WHERE AccountID = 1) < 0
   THEN 'Transaction rolled back! Insufficient funds.'
   ELSE 'Transaction successful!'
END;

ROLLBACK;

SELECT * FROM BankAccounts;
```

Aim :

Table: Accounts

Perform a series of transactions on a banking system(Table named Accounts) . Start by deducting $200 from Alice's account and adding it to Bob's account. Create a SAVEPOINT named AfterAliceDeduction to mark the state after this transfer. Then, transfer $100 from Bob's account to Charlie's account. **Roll**

back to the **SAVEPOINT** AfterAliceDeduction to undo the second transfer while keeping the first one intact. Finally, commit the transaction to save the valid changes and show the new table.

+----------------+-----------+
| Column Name | Type |
+----------------+-----------+
AccountID	int
AccountHolder	varchar
Balance	decimal
+----------------+-----------+

Input Format :

+------------+----------------+---------+
| AccountID | AccountHolder | Balance |
+------------+----------------+---------+
1	Alice	500.00
2	Bob	300.00
3	Charlie	700.00
+------------+----------------+---------+

Output Format :

1|Alice|300
2|Bob|500
3|Charlie|700

Algorithm :

Source Code :

```
BEGIN TRANSACTION;

UPDATE Accounts
SET Balance = Balance - 200
WHERE AccountID = 1;

UPDATE Accounts
SET Balance = Balance + 200
WHERE AccountID = 2;

SAVEPOINT AfterAliceDeduction;

UPDATE Accounts
SET Balance = Balance - 100
WHERE AccountID = 2;

UPDATE Accounts
SET Balance = Balance + 100
WHERE AccountID = 3;

ROLLBACK TO AfterAliceDeduction;

COMMIT;
SELECT * FROM Accounts;
```

Aim :

Table: Accounts

Perform a series of transactions on the Accounts table. Create the Accounts table with columns AccountID, AccountHolder, and Balance. Insert sample data for five accounts: Alice, Bob, Charlie, David, and Eve.

Begin a transaction:

- Deduct $200 from Alice's account and add it to Bob's account.
- Create a SAVEPOINT AfterAliceDeduction.
- Transfer $100 from Bob's account to Charlie's account.

Roll back to After Alice Deduction to undo the second transfer (Bob to Charlie), Commit the transaction and show the final table.

```
+-----------------+-----------+
| Column Name   | Type   |
+-----------------+-----------+
```

```
| AccountID    | int     |
| AccountHolder | varchar |
| Balance      | decimal |
+--------------+---------+
```

Input Format :

```
+-----------+---------------+---------+
| AccountID | AccountHolder | Balance |
+-----------+---------------+---------+
| 1     | Alice     | 1000.00 |
| 2     | Bob       | 1500.00 |
| 3     | Charlie   | 1200.00 |
+-----------+---------------+---------+
```

Output Format :

```
+-----------+---------------+---------+
| AccountID | AccountHolder | Balance |
+-----------+---------------+---------+
| 1     | Alice     | 800.00  |
| 2     | Bob       | 1700.00 |
| 3     | Charlie   | 1200.00 |
+-----------+---------------+---------+
```

Algorithm :

Source Code :

```
-- Step 3: Begin the transaction
BEGIN;

-- Deduct $200 from Alice's account and add it to Bob's account
UPDATE Accounts
SET Balance = Balance - 200
WHERE AccountHolder = 'Alice';

UPDATE Accounts
SET Balance = Balance + 200
WHERE AccountHolder = 'Bob';

-- Step 4: Create a SAVEPOINT to mark the state after the deduction
SAVEPOINT AfterAliceDeduction;

-- Transfer $100 from Bob's account to Charlie's account
UPDATE Accounts
SET Balance = Balance - 100
WHERE AccountHolder = 'Bob';

UPDATE Accounts
SET Balance = Balance + 100
WHERE AccountHolder = 'Charlie';

-- Step 5: Rollback to the SAVEPOINT to undo the second transfer (Bob to Charlie)
ROLLBACK TO SAVEPOINT AfterAliceDeduction;

-- Step 6: Commit the transaction to save the valid changes
COMMIT;

-- Step 7: Show the updated table
SELECT * FROM Accounts;
```

Week 6 (Nested IF, CASE, CASE Expression, NULLIF and COALESCE)

Aim :

Table: Numbers

Write an SQL query to find the greatest of three numbers stored in a table with columns num1, num2, and num3 using the CASE statement. The query should compare the numbers and return the greatest one, along with the original values. The feature being used is the CASE expression, which allows conditional logic to determine the largest value based on comparisons between the columns.

```
+-----------------+-----------+
| Column Name  | Type   |
+-----------------+-----------+
| num1     | INT    |
| num2     | INT    |
| num3     | INT    |
+-----------------+-----------+
```

Input Format :

```
+-----------+-----------+-----------+
| num1  | num2  | num3  |
+-----------+-----------+-----------+
| 10    | 20    | 15    |
| 20    | 30    | 25    |
| 50    | 40    | 60    |
+-----------+-----------+-----------+
```

Output Format :

```
+-----------+-----------+-----------+-----------+
| num1  | num2  | num3  | Greatest Number  |
+-----------+-----------+-----------+-----------+
| 10    | 20    | 15    | 20    |
| 5     | 30    | 25    | 30    |
| 50    | 40    | 60    | 60    |
+-----------+-----------+-----------+-----------+
```

Algorithm :

Source Code :

```
SELECT
    num1,
    num2,
    num3,
    CASE
        WHEN num1 >= num2 AND num1 >= num3 THEN num1
        WHEN num2 >= num1 AND num2 >= num3 THEN num2
        ELSE num3
    END AS GreatestNumber
FROM Numbers;
```

Aim :

Write an SQL query to classify each character from a given set of characters as either a vowel or a consonant using the **CASE** statement combined with the **NULLIF** function. The query should return the character along with its classification.

Table: Characters

```
+-----------------+-------------+
| Column Name  | Type    |
+-----------------+-------------+
| letter    | CHAR(1)   |
+-----------------+-------------+
```

Input Format :

```
+--------+
| letter |
+--------+
| a  |
| b  |
| E  |
| f  |
| I  |
| g  |
```

```
| O |
| p |
| U |
+--------+
```

Output Format :

```
+--------+---------------+
| letter | vowel/consonant|
+--------+---------------+
| a  | Vowel     |
| b  | Consonant  |
| E  | Vowel     |
| f  | Consonant  |
| I  | Vowel     |
| g  | Consonant  |
| O  | Vowel     |
| p  | Consonant  |
| U  | Vowel     |
+--------+---------------+
```

Algorithm :

Source Code :

```
SELECT
  letter,
  CASE
    WHEN NULLIF(letter, 'a') IS NULL OR NULLIF(letter, 'e') IS NULL OR NULLIF(letter, 'i') IS
NULL OR
        NULLIF(letter, 'o') IS NULL OR NULLIF(letter, 'u') IS NULL OR
        NULLIF(letter, 'A') IS NULL OR NULLIF(letter, 'E') IS NULL OR
        NULLIF(letter, 'I') IS NULL OR NULLIF(letter, 'O') IS NULL OR
        NULLIF(letter, 'U') IS NULL THEN 'Vowel'
    ELSE 'Consonant'
  END AS CharacterType
FROM Characters;
```

Aim :

Write an SQL query to replace NULL salary values with a default value of 30000 for employees in a table. Use the COALESCE function to achieve this, and return the employee name along with their salary.

Table: employees

```
+------------------+----------------+
| Column Name   | Type     |
+------------------+----------------+
| employee_id   | INT      |
| employee_name  | VARCHAR(50)   |
| salary     | INT      |
+------------------+----------------+
```

Input Format :

```
+------------------+----------------+-------------+
| employee_id   | employee_name  | salary   |
+------------------+----------------+-------------+
| 1      | 'Alice'    | 50000    |
| 2      | 'Bob'     | NULL    | -- Bob has no salary (NULL)
| 3      | 'Charlie'    | 70000    |
| 4      | 'David'    | NULL    | -- David has no salary (NULL)
| 5      | 'Eve'     | 40000    |
+------------------+----------------+-------------+
```

Output Format :

```
+-----------+---------+
| employee_name | salary |
+-----------+---------+
| Alice     | 50000 |
| Bob      | 30000 |
| Charlie    | 70000 |
| David     | 30000 |
| Eve      | 40000  |
+-----------+---------+
```

Algorithm :

Source Code :

```
SELECT
  employee_name,
  COALESCE(salary, 30000) AS salary
FROM employees;
```

Aim :

Table: employees

Write an SQL query using the **CASE expression** to classify employees into salary categories: **"High Salary" for salaries >= 60000**, **"Medium Salary" for salaries >= 40000**, and **"Low Salary" for salaries below 40000.** Return the employee name, salary, and their salary category.

```
+------------------+-----------------+
| Column Name   | Type     |
+------------------+-----------------+
| employee_id   | INT      |
| employee_name  | VARCHAR(50)   |
| salary     | INT      |
+------------------+-----------------+
```

Input Format :

```
+------------------+-----------------+-------------+
| employee_id   | employee_name  | salary   |
+------------------+-----------------+-------------+
| 1      | 'Sophia'    | 45000    |
| 2      | 'Jackson'    | 25000    |
| 3      | 'Oliver'    | 60000    |
| 4      | 'Amelia'    | 35000    |
| 5      | 'Liam'     | 70000    |
+------------------+-----------------+-------------+
```

Output Format :

Sophia|45000|Medium Salary

Jackson|25000|Low Salary

Oliver|60000|High Salary

Amelia|35000|Low Salary

Liam|70000|High Salary

Algorithm :

Source Code :

```
SELECT
  employee_name,
  salary,
  CASE
    WHEN salary >= 60000 THEN 'High Salary'
    WHEN salary >= 40000 THEN 'Medium Salary'
    ELSE 'Low Salary'
  END AS salary_category
FROM employees;
```

Aim :

Table: employees

Problem Statement: **Employee Bonus Eligibility**

You are given an employee's **performance rating** and **years of experience**. Write an SQL query to determine if the employee is eligible for a bonus and the bonus percentage, based on the following conditions:

1.Excellent performance (rating = 5) qualifies for:

- **10% bonus** if the employee has **less than 5 years of experience**.
- **15% bonus** if the employee has **5 or more years of experience**.

2.Good performance (rating = 4) qualifies for:

- **5% bonus** if the employee has **less than 5 years of experience**.
- **10% bonus** if the employee has **5 or more years of experience**.

3.performance (rating = 3) or lower does **not qualify for a bonus**.

+----------------------------+----------------+

| Column Name | Type |

+----------------------------+----------------+

|employee_id | INT|

| **rating** | INT |

| **years_of_experience** | INT |

+----------------------------+----------------+

Input Format :

Input:

employee_id (integer) **rating** (integer) **years_of_experience** (integer)

(1, 5, 3)

(2, 5, 5)

(3, 4, 2)

(4, 4, 6)

(5, 3, 4)

(6, 2, 8)

(7, 5, 0)

(8, 4, 0)

(9, 1, 10)

(10, 5, 15)

Output Format :

Output:

1|5|3|10% Bonus

2|5|5|15% Bonus

3|4|2|5% Bonus

4|4|6|10% Bonus

5|3|4|No Bonus

6|2|8|No Bonus

7|5|0|10% Bonus

8|4|0|5% Bonus

9|1|10|No Bonus

10|5|15|15% Bonus

Algorithm :

Source Code :

```
SELECT
  employee_id,
  performance_rating,
  years_of_experience,
  CASE
    WHEN performance_rating = 5 THEN
      CASE
        WHEN years_of_experience < 5 THEN '10% Bonus'
        ELSE '15% Bonus'
      END
    WHEN performance_rating = 4 THEN
      CASE
        WHEN years_of_experience < 5 THEN '5% Bonus'
        ELSE '10% Bonus'
      END
    ELSE 'No Bonus'
  END AS bonus
FROM employees;
```

Week 7(Loops, Error Handling, Exceptions)

Aim :

Table: multiplication_table

Write a query to generate the multiplication table for a given number N (from 1 to 10) using For Loop?

Input Format :

Input:

5

Output Format :

5 x 1 = 5

5 x 2 = 10

5 x 3 = 15

5 x 4 = 20

5 x 5 = 25

5 x 6 = 30

5 x 7 = 35

5 x 8 = 40

5 x 9 = 45

5 x 10 = 50

Source Code:

```
DO $$
DECLARE
    fetched_num INTEGER;
BEGIN
    -- Fetch the value from input_value table
    SELECT num INTO fetched_num FROM input_value;

    -- Loop to generate multiples of fetched_num
    FOR counter IN 1..10 LOOP
        INSERT INTO multiples_output (value) VALUES (fetched_num * counter);
    END LOOP;
END $$;

-- Retrieve the stored results
SELECT * FROM multiples_output;
```

Aim :

Use **FOR Loop**

PL/SQL CODE TO Print 10 multiples from 10 to 100

Note:

Write a PostgreSQL script that:

1. Creates a temporary table **multiples_output** to store multiples of a number.
2. Creates an input table **input_value** to store a single integer value.
3. Inserts a test value into the **input_value** table.
4. Uses a PL/pgSQL block to:
5. Retrieve the input value from **input_value**.
6. Generate and store the first 10 multiples of the retrieved value in **multiples_output**.
7. Selects and displays the stored multiples from **multiples_output**.

Input Format :

10

Output Format :

value

10

20

30

40

50

60

70

80

90

100

Algorithm :

Source Code :

```
DO $$
DECLARE
    fetched_num INTEGER;
BEGIN
    -- Fetch the value from input_value table
    SELECT num INTO fetched_num FROM input_value;

    -- Loop to generate multiples of fetched_num
    FOR counter IN 1..10 LOOP
        INSERT INTO multiples_output (value) VALUES (fetched_num * counter);
    END LOOP;
END $$;

-- Retrieve the stored results
SELECT * FROM multiples_output;
```

Aim :

Write a query to generate the multiplication table for a given number N (from 1 to 10) using while loop

Input Format :

Input:

5

Output Format :

5 x 1 = 5

5 x 2 = 10

5 x 3 = 15

5 x 4 = 20

5 x 5 = 25

5 x 6 = 30

5 x 7 = 35

5 x 8 = 40

5 x 9 = 45

5 x 10 = 50

Source Code:

```
DO $$
DECLARE
    start_val INTEGER;
    end_val INTEGER;
    num INTEGER;
    div INTEGER;
    is_prime BOOLEAN;
BEGIN
    -- Fetch start and end values from test_case table
    SELECT start_num, end_num INTO start_val, end_val FROM test_case;

    -- Loop through numbers from start_val to end_val
    FOR num IN start_val..end_val LOOP
        is_prime := TRUE; -- Assume the number is prime initially

        -- Check divisibility from 2 to num-1
        FOR div IN 2..num-1 LOOP
            IF num % div = 0 THEN
                is_prime := FALSE; -- Not a prime number
                EXIT; -- No need to check further
            END IF;
        END LOOP;

        -- If number is prime, insert it into prime_numbers table
        IF is_prime THEN
            INSERT INTO prime_numbers (value) VALUES (num);
        END IF;
    END LOOP;
END $$;

-- Retrieve and display the prime numbers
SELECT * FROM prime_numbers;
```

Aim :

While Loop

Code for PL/SQL While Loop to print 1 to 10 numbers?

Note:

Write a PostgreSQL script that:

1. Creates a temporary table **loop_output** to store a sequence of numbers.
2. Creates a table **single_value** to store a single integer value.
3. Inserts a test value into the **single_value** table.
4. Uses a PL/pgSQL block to:
5. Retrieve the input value from **single_value**.
6. Use a **WHILE loop** to insert numbers from **1 to the retrieved value** into **loop_output**.
7. Selects and displays the stored numbers from **loop_output**.

Input Format :

10

Output Format :

value

1

2

3

4

5

6

7

8

9
10

Algorithm :

Source Code :

```
DO $$
DECLARE
   fetched_num INTEGER;
   counter INTEGER := 1;
BEGIN
   -- Fetch the value from single_value table
   SELECT num INTO fetched_num FROM single_value;


   -- Loop from 1 to fetched_num and store results in loop_output table
   WHILE counter <= fetched_num LOOP
      INSERT INTO loop_output (value) VALUES (counter);
      counter := counter + 1;
   END LOOP;
END $$;


-- Retrieve the stored results
SELECT * FROM loop_output;
```

Aim :

Nested Loops

PL/SQL CODE uses a nested basic loop to find the prime numbers from 2 to 50

Note:

Write a PostgreSQL script that:

1. Creates a temporary table **test_case** to store a range of numbers for prime number searching.

2. Inserts a test range into **test_case** with a **start number (2)** and an **end number (50)**.
3. Creates a temporary table **prime_numbers** to store the identified prime numbers.
4. Uses a PL/pgSQL block to:
5. Retrieve the start and end values from **test_case**.
6. Iterate through each number in the range and determine if it is **prime**.
7. If a number is prime, insert it into the **prime_numbers** table.
8. Selects and displays the stored prime numbers from **prime_numbers**.

Input Format :

2

50

Output Format :

value

2

3

5

7

11

13

17

19

23

29

31

37

41

43

47

Algorithm :

Source Code :

```
DO $$
DECLARE
    start_val INTEGER;
    end_val INTEGER;
    num INTEGER;
    div INTEGER;
    is_prime BOOLEAN;
BEGIN
    -- Fetch start and end values from test_case table
    SELECT start_num, end_num INTO start_val, end_val FROM test_case;

    -- Loop through numbers from start_val to end_val
    FOR num IN start_val..end_val LOOP
        is_prime := TRUE; -- Assume the number is prime initially

        -- Check divisibility from 2 to num-1
        FOR div IN 2..num-1 LOOP
            IF num % div = 0 THEN
                is_prime := FALSE; -- Not a prime number
                EXIT; -- No need to check further
            END IF;
        END LOOP;

        -- If number is prime, insert it into prime_numbers table
        IF is_prime THEN
            INSERT INTO prime_numbers (value) VALUES (num);
        END IF;
    END LOOP;
END $$;

-- Retrieve and display the prime numbers
SELECT * FROM prime_numbers;
```

Aim :

problem statement :

Create a new table student1 that contains only unique records from the existing student table, while identifying and logging duplicate records using exception handling in PostgreSQL.

```
+------------------+--------------------------+
| Column Name    | Type          |
+------------------+--------------------------+
| id     | INTEGER PRIMARY KEY    |
```

| name | VARCHAR(50) |
+------------------+--------------------------+

Input Format :

input format :

Student

+---------------+--------------------------+
| id | name |
+---------------+--------------------------+
| 1 | vikram |
+---------------+--------------------------+
| 2 | jhon |
+---------------+--------------------------+
| 3 | thor |
+---------------+--------------------------+
| 4 | parker |
+---------------+--------------------------+
| 1 | vikram |
+---------------+--------------------------+
| 4 | parker |
+---------------+--------------------------+
| 1 | vikram |
+---------------+--------------------------+

Output Format :

output format :

vikram
parker
vikram
1 vikram
2 jhon
3 thor
4 parker

Algorithm :

Source Code :

```
-- Creating the student1 table with only unique records
CREATE TABLE student1 AS
SELECT DISTINCT ON (name) * FROM student
ORDER BY name;

-- Print unique names (from student1)
DO $$
DECLARE
   r RECORD;
BEGIN
   FOR r IN SELECT name FROM student1 ORDER BY name
   LOOP
      RAISE NOTICE '%', r.name;  -- This will print the unique names
   END LOOP;
END $$;

-- Print all records (from student table)
DO $$
DECLARE
   r RECORD;
BEGIN
   FOR r IN SELECT id, name FROM student1 ORDER BY id
   LOOP
      RAISE NOTICE '% %', r.id, r.name;  -- This will print all records
   END LOOP;
END $$;
```

Week - 8 (Procedures)

Aim :

Code to find Minimum of two numbers using findMin() Procedure using Postgresql?

Note:

Write a PostgreSQL script that:

- Creates a temporary table **min_value_output** to store the minimum value of two numbers.
- Creates an input table **input_values** to store two integer values.
- Inserts a test pair of values into the **input_values** table.
- Uses a PL block to:
- Retrieve the input values from **input_values**.
- Call a stored procedure findMin() to determine the minimum of the two numbers.
- Store the result in **min_value_output.**
- Selects and displays the stored minimum value from **min_value_output.**

Input Format :

25, 8

Output Format :

min_value

500

Algorithm :

Source Code :

```
DROP PROCEDURE IF EXISTS findMin;

CREATE OR REPLACE PROCEDURE findMin(IN num1 INT, IN num2 INT, OUT minValue INT)
LANGUAGE plpgsql
AS $$
BEGIN
    minValue := LEAST(num1, num2);
END;
$$;

DO $$
DECLARE
   fetched_num1 INTEGER;
   fetched_num2 INTEGER;
   result INTEGER;
BEGIN
   SELECT num1, num2 INTO fetched_num1, fetched_num2 FROM input_values;

   CALL findMin(fetched_num1, fetched_num2, result);

   INSERT INTO min_value_output (min_value) VALUES (result);
END $$;

SELECT * FROM min_value_output;
```

Week9(Functions)

Aim :

create a Function which counts no.of Employees in Employee table

+--------------+-----------+

| Column Name | Type |

+--------------+-----------+

| EMPN | int |

| ENAME | varchar |

| JOB | varchar |

| HIREDATE | date |

| SALARY | float |

| DEPTNO | int |

+--------------+-----------+

Each row of this table contains the EMPN, ENAME, JOB, HIREDATE, SALAR, and DEPTNO of an employee.

Input Format :

+------+--------+-----------+------------+-------+-------+

| EMPN | ENAME | JOB | HIREDATE | SALARY | DEPTNO|

+------+--------+-----------+------------+-------+-------+

| 7369 | SMITH | CLERK | 17-DEC-80 | 800 | 20 |

| 7499 | ALLEN | SALESMAN | 20-FEB-81 | 1600 | 30 |

| 7521 | WARD | SALESMAN | 22-FEB-81 | 1250 | 30 |

| 7566 | JONES | MANAGER | 02-APR-81 | 2975 | 20 |

| 7654 | MARTIN | SALESMAN | 28-SEP-81 | 1250 | 30 |

| 7698 | BLAKE | MANAGER | 01-MAY-81 | 2850 | 30 |

| 7782 | CLARK | MANAGER | 09-JUN-81 | 2450 | 10 |

| 7788 | SCOTT | ANALYST | 19-APR-87 | 3000 | 20 |

| 7839 | KING | PRESIDENT | 17-NOV-81 | 5000 | 10 |

| 7844 | TURNER | SALESMAN | 08-SEP-81 | 1500 | 30 |

| 7876 | ADAMS | CLERK | 23-MAY-87 | 1100 | 20 |

| 7900 | JAMES | CLERK | 03-DEC-81 | 950 | 30 |

| 7902 | FORD | ANALYST | 03-DEC-81 | 3000 | 20 |

| 7934 | MILLER | CLERK | 23-JAN-82 | 1300 | 10 |

+------+--------+-----------+------------+-------+-------+

Output Format :

EmployeeCount

14

Algorithm :

Source Code :

```
-- Creating the function to count employees
CREATE OR REPLACE FUNCTION CountEmployees()
RETURNS INT AS $$
DECLARE
    EmployeeCount INT;
BEGIN
    -- Counting the number of employees
    SELECT COUNT(*) INTO EmployeeCount
    FROM Employee;

    RETURN EmployeeCount;
END;
$$ LANGUAGE plpgsql;

-- Calling the function to get the total number of employees
SELECT CountEmployees() AS EmployeeCount;
```

Week 10 & 11(Triggers)

Aim :

BEFORE INSERT Trigger: This trigger should ensure that whenever a new user is inserted into the users table, the email is automatically converted to lowercase before the insertion. For example, if the email "Alex@Example.Com" is inserted, it should be converted to "alex@example.com".
AFTER UPDATE Trigger: This trigger should log any email changes in the user_log table. The log should store the user_id, the old email (OLD.email), and the new email (NEW.email) whenever an update occurs. For example, if the following SQL command is executed:

```
+------------------+--------------------------+
| Column Name    | Type          |
+------------------+--------------------------+
| user_id     | INTEGER PRIMARY KEY    |
| username     | VARCHAR(50)       |
| email      | VARCHAR(100)      |
+------------------+--------------------------+
```

```
+------------------+--------------------------+
| Column Name    | Type          |
+------------------+--------------------------+
| log_id     | INTEGER PRIMARY KEY AUTOINCREMENT |
| user_id     | INTEGER         |
| old_email     | VARCHAR(100)      |
| new_email     | VARCHAR(100)      |
+------------------+--------------------------+
```

Input Format :

```
+---------+--------------+---------------------+
| user_id | username   | email       |
+---------+--------------+---------------------+
| 1   | john_doe   | john@example.com  |
| 2   | jane_doe   | jane@example.com  |
| 3   | alex_smith  | alex@example.com  |
+---------+--------------+---------------------+
```

Output Format :

```
+--------+---------+------------------+--------------------+
| log_id | user_id | old_email     | new_email     |
```

```
+--------+---------+------------------+--------------------+
| 1   | 3    | alex@example.com | alexnew@example.com |
+--------+---------+------------------+--------------------+
```

Algorithm :

Source Code :

```
CREATE TRIGGER before_user_insert
BEFORE INSERT ON users
FOR EACH ROW
BEGIN
   -- Convert email to lowercase before insertion
   UPDATE users
   SET email = LOWER(NEW.email)
   WHERE user_id = NEW.user_id;
END;

-- 4. Create the AFTER UPDATE Trigger
CREATE TRIGGER after_user_update
AFTER UPDATE ON users
FOR EACH ROW
BEGIN
   INSERT INTO user_log (user_id, old_email, new_email)
   VALUES (OLD.user_id, OLD.email, NEW.email);  -- Log old and new email
END;
UPDATE users
SET email = 'AlexNew@example.Com'
WHERE user_id = 3;

-- 9. Check the User Log Table (The log should contain the old and new emails, without time)
SELECT * FROM user_log;
```

Aim :

Create a row-level trigger that updates the last_updated column to 'Salary Updated' whenever the salary of an employee is updated.
Demonstrate the functionality by updating the salary of an employee (e.g., update the salary of the employee with id = 1 to 55000.00).
View the updated table to verify the last_updated column is correctly updated.

```
+------------------+-------------------------+
| Column Name    | Type          |
+------------------+-------------------------+
| id       | INT PRIMARY KEY     |
| name       | VARCHAR(100)     |
| salary     | DECIMAL(10, 2)     |
| last_updated   | VARCHAR(255)     |
+------------------+-------------------------+
```

Input Format :

```
+----+------------------+---------+------------------+
| id | name      | salary | last_updated   |
+----+------------------+---------+------------------+
| 1 | John Doe     | 50000.00| Record Created  |
| 2 | Jane Smith    | 60000.00| Record Created  |
| 3 | Alice Johnson   | 55000.00| Record Created  |
+----+------------------+---------+------------------+
```

Output Format :

1|John Doe|55000|Salary Updated
2|Jane Smith|60000|Record Created
3|Alice Johnson|55000|Record Created

Algorithm :

Source Code :

```
CREATE TRIGGER update_employee_note
BEFORE UPDATE ON employees
FOR EACH ROW
BEGIN
   UPDATE employees
   SET last_updated = 'Salary Updated'
   WHERE id = OLD.id;
END;

UPDATE employees
SET salary = 55000.00
WHERE id = 1;

SELECT * FROM employees;
```

Week 12 (Search Operations)

Aim :

Create a Customers table to store customer details such as CustomerID, Name, Email, City, and Phone. Insert sample data into the table and create an index on the City column to optimize search queries. Finally, write a query to retrieve all customers who reside in a specific city, such as New York.

```
+-----------------+-------------------+
| Column Name  | Type      |
+-----------------+-------------------+
| CustomerID   | INT PRIMARY KEY  |
| Name     | VARCHAR(50)    |
| Email    | VARCHAR(50)    |
| City     | VARCHAR(50)    |
| Phone    | VARCHAR(15)    |
+-----------------+-------------------+
```

Input Format :

```
+-------------+--------------+--------------------+-------------+------------+
| CustomerID | Name    | Email       | City    | Phone   |
+-------------+--------------+--------------------+-------------+------------+
| 1     | Alice Smith | alice@example.com  | New York  | 1234567890 |
| 2     | Bob Johnson | bob@example.com   | Los Angeles | 2345678901 |
| 3     | Charlie Davis| charlie@example.com | New York  | 3456789012 |
| 4     | Diana White | diana@example.com  | Chicago   | 4567890123 |
| 5     | Edward Brown | edward@example.com  | Los Angeles | 5678901234 |
+-------------+--------------+--------------------+-------------+------------+
```

Output Format :

1|Alice Smith|alice@example.com|New York|1234567890

3|Charlie Davis|charlie@example.com|New York|3456789012

Algorithm :

Source Code :

```
CREATE INDEX idx_city ON Customers (City);

SELECT * FROM Customers WHERE City = 'New York';
```

Aim :

Write a query to manage customer orders by creating an Orders table to store details such as OrderID, CustomerName, OrderDate, Product, and Quantity. Populate the table with sample data, and perform a non-indexing search to retrieve all orders where the product is a 'Laptop'.

```
+-----------------+-------------------+
| Column Name   | Type      |
+-----------------+-------------------+
| OrderID     | INT PRIMARY KEY  |
| CustomerName  | VARCHAR(50)    |
| OrderDate    | DATE      |
| Product     | VARCHAR(50)    |
| Quantity    | INT       |
+-----------------+-------------------+
```

Input Format :

```
+---------+---------------+------------+---------+----------+
| OrderID | CustomerName | OrderDate | Product | Quantity |
+---------+---------------+------------+---------+----------+
| 1   | Alice Smith  | 2025-01-01| Laptop | 2    |
| 2   | Bob Johnson  | 2025-01-03| Phone  | 1    |
| 3   | Charlie Davis | 2025-01-04| Tablet | 3    |
| 4   | Diana White  | 2025-01-05| Laptop | 1    |
| 5   | Edward Brown | 2025-01-06| Phone  | 2    |
+---------+---------------+------------+---------+----------+
```

Output Format :

1|Alice Smith|2025-01-01|Laptop|2
4|Diana White|2025-01-05|Laptop|1

Algorithm :

Source Code :

```
SELECT * FROM Orders WHERE Product = 'Laptop';
```

week 13

Aim :

Write a Java program that connects to a database using JDBC

File Name :

pom.xml

Code :

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
http://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>

  <groupId>com.java.workspace</groupId>
  <artifactId>java-workspace</artifactId>
  <version>1.0-SNAPSHOT</version>

  <name>java-workspace</name>
  <!-- FIXME change it to the project's website -->
  <url>http://www.example.com</url>

  <properties>
    <project.build.sourceEncoding>UTF-
8</project.build.sourceEncoding>
    <maven.compiler.release>17</maven.compiler.release>
  </properties>

  <dependencyManagement>
    <dependencies>
      <dependency>
        <groupId>org.junit</groupId>
        <artifactId>junit-bom</artifactId>
        <version>5.11.0</version>
        <type>pom</type>
        <scope>import</scope>
      </dependency>
```

```
    </dependencies>
  </dependencyManagement>

  <dependencies>
    <dependency>
      <groupId>org.junit.jupiter</groupId>
      <artifactId>junit-jupiter-api</artifactId>
      <scope>test</scope>
    </dependency>
    <!-- Optionally: parameterized tests support -->
    <dependency>
      <groupId>org.junit.jupiter</groupId>
      <artifactId>junit-jupiter-params</artifactId>
      <scope>test</scope>
    </dependency>

    <!-- https://mvnrepository.com/artifact/com.mysql/mysql-
connector-j -->
    <dependency>
        <groupId>com.mysql</groupId>
        <artifactId>mysql-connector-j</artifactId>
        <version>9.2.0</version>
    </dependency>
  </dependencies>

  <build>
    <pluginManagement><!-- lock down plugins versions to avoid
using Maven defaults (may be moved to parent pom) -->
      <plugins>
        <!-- clean lifecycle, see
https://maven.apache.org/ref/current/maven-
core/lifecycles.html#clean_Lifecycle -->
        <plugin>
          <artifactId>maven-clean-plugin</artifactId>
          <version>3.4.0</version>
        </plugin>
        <!-- default lifecycle, jar packaging: see
https://maven.apache.org/ref/current/maven-core/default-
bindings.html#Plugin_bindings_for_jar_packaging -->
        <plugin>
          <artifactId>maven-resources-plugin</artifactId>
          <version>3.3.1</version>
        </plugin>
```

```
        <plugin>
          <artifactId>maven-compiler-plugin</artifactId>
          <version>3.13.0</version>
        </plugin>
        <plugin>
          <artifactId>maven-surefire-plugin</artifactId>
          <version>3.3.0</version>
        </plugin>
        <plugin>
          <artifactId>maven-jar-plugin</artifactId>
          <version>3.4.2</version>
        </plugin>
        <plugin>
          <artifactId>maven-install-plugin</artifactId>
          <version>3.1.2</version>
        </plugin>
        <plugin>
          <artifactId>maven-deploy-plugin</artifactId>
          <version>3.1.2</version>
        </plugin>
        <!-- site lifecycle, see
https://maven.apache.org/ref/current/maven-
core/lifecycles.html#site_Lifecycle -->
        <plugin>
          <artifactId>maven-site-plugin</artifactId>
          <version>3.12.1</version>
        </plugin>
        <plugin>
          <artifactId>maven-project-info-reports-
plugin</artifactId>
          <version>3.6.1</version>
        </plugin>
      </plugins>
    </pluginManagement>
  </build>
</project>
```

File Name :

src/main/java/com/java/workspace/App.java

Code :

```
package com.java.workspace;

/**
 * Hello world!
 */
public class App {
    public static void main(String[] args) {
        System.out.println("Hello World!");
    }
}
```

File Name :

src/main/java/com/java/workspace/JDBCOrders.java

Code :

```
package com.java.workspace;


import java.sql.*;


public class JDBCOrders {
    public static void main(String[] args) {
        // Database connection details
        String url = "jdbc:mysql://localhost:3306/workspace";
// Change database name if needed
        String user = "root"; // Your MySQL username
        String password = "root"; // Your MySQL password


        Connection conn = null;
        Statement stmt = null;


        try {
            // 1. Load MySQL JDBC Driver
```

```
            Class.forName("com.mysql.cj.jdbc.Driver");


            // 2. Establish a connection
            conn = DriverManager.getConnection(url, user,
password);
            System.out.println("Connected to the database
successfully!");


            // 3. Create a statement
            stmt = conn.createStatement();


            // 4. Create the Orders table if it doesn't exist
            String createTableSQL = "CREATE TABLE IF NOT EXISTS
Orders ("
                    + "OrderID INT PRIMARY KEY AUTO_INCREMENT,
"
                    + "CustomerName VARCHAR(50), "
                    + "OrderDate DATE, "
                    + "Product VARCHAR(50), "
                    + "Quantity INT)";
            stmt.executeUpdate(createTableSQL);
            System.out.println("Orders table created or already
exists.");


            // 5. Insert sample data (if the table is empty)
            String checkDataSQL = "SELECT COUNT(*) FROM
Orders";
            ResultSet rs = stmt.executeQuery(checkDataSQL);
            rs.next();
            int rowCount = rs.getInt(1);

            if (rowCount == 0) { // Only insert data if table
is empty
                String insertDataSQL = "INSERT INTO Orders
(CustomerName, OrderDate, Product, Quantity) VALUES "
                        + "('Alice Smith', '2025-01-01',
'Laptop', 2), "
                        + "('Bob Johnson', '2025-01-03',
'Phone', 1), "
```

```
                        + "('Charlie Davis', '2025-01-04',
'Tablet', 3), "
                        + "('Diana White', '2025-01-05',
'Laptop', 1), "
                        + "('Edward Brown', '2025-01-06',
'Phone', 2)";
                stmt.executeUpdate(insertDataSQL);
                System.out.println("Sample data inserted into
Orders table.");
            } else {
                System.out.println("Orders table already
contains data.");
            }


            // 6. Retrieve and display orders where Product is
'Laptop'
            String selectSQL = "SELECT * FROM Orders WHERE
Product = 'Laptop'";
            rs = stmt.executeQuery(selectSQL);


            System.out.println("\nOrders with Product =
'Laptop':");
            while (rs.next()) {
                int orderId = rs.getInt("OrderID");
                String customerName =
rs.getString("CustomerName");
                String orderDate = rs.getString("OrderDate");
                String product = rs.getString("Product");
                int quantity = rs.getInt("Quantity");


                System.out.println(orderId + " | " +
customerName + " | " + orderDate + " | " + product + " | " +
quantity);
            }
        } catch (Exception e) {
            e.printStackTrace();
        } finally {
            try {
                if (stmt != null) stmt.close();
                if (conn != null) conn.close();
```

```
            } catch (SQLException se) {
                se.printStackTrace();
            }
        }
    }
}
```

File Name :

src/test/java/com/java/workspace/AppTest.java

Code :

```
package com.java.workspace;

import static org.junit.jupiter.api.Assertions.assertTrue;

import org.junit.jupiter.api.Test;

/**
 * Unit test for simple App.
 */
public class AppTest {

    /**
     * Rigorous Test :-)
     */
    @Test
    public void shouldAnswerWithTrue() {
        assertTrue(true);
    }
}
```

File Name :

```
    </dependencies>
  </dependencyManagement>

  <dependencies>
    <dependency>
      <groupId>org.junit.jupiter</groupId>
      <artifactId>junit-jupiter-api</artifactId>
      <scope>test</scope>
    </dependency>
    <!-- Optionally: parameterized tests support -->
    <dependency>
      <groupId>org.junit.jupiter</groupId>
      <artifactId>junit-jupiter-params</artifactId>
      <scope>test</scope>
    </dependency>

    <!-- https://mvnrepository.com/artifact/com.mysql/mysql-
connector-j -->
    <dependency>
        <groupId>com.mysql</groupId>
        <artifactId>mysql-connector-j</artifactId>
        <version>9.2.0</version>
    </dependency>
  </dependencies>

  <build>
    <pluginManagement><!-- lock down plugins versions to avoid
using Maven defaults (may be moved to parent pom) -->
      <plugins>
        <!-- clean lifecycle, see
https://maven.apache.org/ref/current/maven-
core/lifecycles.html#clean_Lifecycle -->
        <plugin>
          <artifactId>maven-clean-plugin</artifactId>
          <version>3.4.0</version>
        </plugin>
        <!-- default lifecycle, jar packaging: see
https://maven.apache.org/ref/current/maven-core/default-
bindings.html#Plugin_bindings_for_jar_packaging -->
        <plugin>
          <artifactId>maven-resources-plugin</artifactId>
          <version>3.3.1</version>
        </plugin>
```

```
        <plugin>
          <artifactId>maven-compiler-plugin</artifactId>
          <version>3.13.0</version>
        </plugin>
        <plugin>
          <artifactId>maven-surefire-plugin</artifactId>
          <version>3.3.0</version>
        </plugin>
        <plugin>
          <artifactId>maven-jar-plugin</artifactId>
          <version>3.4.2</version>
        </plugin>
        <plugin>
          <artifactId>maven-install-plugin</artifactId>
          <version>3.1.2</version>
        </plugin>
        <plugin>
          <artifactId>maven-deploy-plugin</artifactId>
          <version>3.1.2</version>
        </plugin>
        <!-- site lifecycle, see
https://maven.apache.org/ref/current/maven-
core/lifecycles.html#site_Lifecycle -->
        <plugin>
          <artifactId>maven-site-plugin</artifactId>
          <version>3.12.1</version>
        </plugin>
        <plugin>
          <artifactId>maven-project-info-reports-
plugin</artifactId>
          <version>3.6.1</version>
        </plugin>
      </plugins>
    </pluginManagement>
  </build>
</project>
```

File Name :

src/main/java/com/java/workspace/App.java

Code :

```
package com.java.workspace;

/**
 * Hello world!
 */
public class App {
    public static void main(String[] args) {
        System.out.println("Hello World!");
    }
}
```

File Name :

src/main/java/com/java/workspace/OrderDatabaseManager.java

Code :

```
package com.java.workspace;


import java.sql.*;


public class OrderDatabaseManager {
    public static void main(String[] args) {
        // Database connection details
        String url = "jdbc:mysql://localhost:3306/workspace";
// Replace with your DB name
        String user = "root"; // Replace with your MySQL
username
        String password = "root"; // Replace with your MySQL
password


        Connection conn = null;
        Statement stmt = null;
        PreparedStatement pstmt = null;
        ResultSet rs = null;
```

```
        try {
            // Load MySQL JDBC Driver
            Class.forName("com.mysql.cj.jdbc.Driver");


            // Establish connection
            conn = DriverManager.getConnection(url, user,
password);
            System.out.println("Connected to the database
successfully!");


            // Step 1: Create Orders Table if it doesn't exist
            stmt = conn.createStatement();
            String createTableSQL = "CREATE TABLE IF NOT EXISTS
Orders (" +
                                    "OrderID INT AUTO_INCREMENT
PRIMARY KEY, " +
                                    "CustomerName VARCHAR(50),
" +
                                    "OrderDate DATE, " +
                                    "Product VARCHAR(50), " +
                                    "Quantity INT)";
            stmt.executeUpdate(createTableSQL);
            System.out.println("Orders table is ready!");


            // Step 2: Insert Sample Data
            String insertSQL = "INSERT INTO Orders
(CustomerName, OrderDate, Product, Quantity) VALUES " +
                               "(?, ?, ?, ?)";
            pstmt = conn.prepareStatement(insertSQL);


            // Sample Data
            String[][] sampleOrders = {
                {"Alice Smith", "2025-01-01", "Laptop", "2"},
                {"Bob Johnson", "2025-01-03", "Phone", "1"},
                {"Charlie Davis", "2025-01-04", "Tablet", "3"},
                {"Diana White", "2025-01-05", "Laptop", "1"},
                {"Edward Brown", "2025-01-06", "Phone", "2"}
```

```
            };


            for (String[] order : sampleOrders) {
                pstmt.setString(1, order[0]); // Customer Name
                pstmt.setDate(2, Date.valueOf(order[1])); //
Order Date
                pstmt.setString(3, order[2]); // Product
                pstmt.setInt(4, Integer.parseInt(order[3])); //
Quantity
                pstmt.executeUpdate();
            }
            System.out.println("Sample orders inserted
successfully!");


            // Step 3: Fetch and Display Data to Verify
Insertion
            String selectSQL = "SELECT * FROM Orders";
            rs = stmt.executeQuery(selectSQL);


            System.out.println("\n=== Order Records ===");
            while (rs.next()) {
                System.out.println("OrderID: " +
rs.getInt("OrderID") +
                                   ", Customer: " +
rs.getString("CustomerName") +
                                   ", Date: " +
rs.getDate("OrderDate") +
                                   ", Product: " +
rs.getString("Product") +
                                   ", Quantity: " +
rs.getInt("Quantity"));
            }


        } catch (Exception e) {
            e.printStackTrace();
        } finally {
            // Close resources
            try {
                if (rs != null) rs.close();
```

```
                if (stmt != null) stmt.close();
                if (pstmt != null) pstmt.close();
                if (conn != null) conn.close();
            } catch (SQLException se) {
                se.printStackTrace();
            }
        }
    }
}
```

File Name :

src/test/java/com/java/workspace/AppTest.java

Code :

```
package com.java.workspace;

import static org.junit.jupiter.api.Assertions.assertTrue;

import org.junit.jupiter.api.Test;

/**
 * Unit test for simple App.
 */
public class AppTest {

    /**
     * Rigorous Test :-)
     */
    @Test
    public void shouldAnswerWithTrue() {
        assertTrue(true);
    }
}
```

File Name :

target/classes/com/java/workspace/App.class

Aim :

Write a Java program to connect to a database using JDBC and delete values from it

File Name :

pom.xml

Code :

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
http://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>

  <groupId>com.java.workspace</groupId>
  <artifactId>java-workspace</artifactId>
  <version>1.0-SNAPSHOT</version>

  <name>java-workspace</name>
  <!-- FIXME change it to the project's website -->
  <url>http://www.example.com</url>

  <properties>
    <project.build.sourceEncoding>UTF-
8</project.build.sourceEncoding>
    <maven.compiler.release>17</maven.compiler.release>
  </properties>

  <dependencyManagement>
    <dependencies>
      <dependency>
        <groupId>org.junit</groupId>
        <artifactId>junit-bom</artifactId>
        <version>5.11.0</version>
        <type>pom</type>
        <scope>import</scope>
      </dependency>
```

```
    </dependencies>
  </dependencyManagement>

  <dependencies>
    <dependency>
      <groupId>org.junit.jupiter</groupId>
      <artifactId>junit-jupiter-api</artifactId>
      <scope>test</scope>
    </dependency>
    <!-- Optionally: parameterized tests support -->
    <dependency>
      <groupId>org.junit.jupiter</groupId>
      <artifactId>junit-jupiter-params</artifactId>
      <scope>test</scope>
    </dependency>

    <!-- https://mvnrepository.com/artifact/com.mysql/mysql-
connector-j -->
    <dependency>
        <groupId>com.mysql</groupId>
        <artifactId>mysql-connector-j</artifactId>
        <version>9.2.0</version>
    </dependency>
  </dependencies>

  <build>
    <pluginManagement><!-- lock down plugins versions to avoid
using Maven defaults (may be moved to parent pom) -->
      <plugins>
        <!-- clean lifecycle, see
https://maven.apache.org/ref/current/maven-
core/lifecycles.html#clean_Lifecycle -->
        <plugin>
          <artifactId>maven-clean-plugin</artifactId>
          <version>3.4.0</version>
        </plugin>
        <!-- default lifecycle, jar packaging: see
https://maven.apache.org/ref/current/maven-core/default-
bindings.html#Plugin_bindings_for_jar_packaging -->
        <plugin>
          <artifactId>maven-resources-plugin</artifactId>
          <version>3.3.1</version>
        </plugin>
```

```
        <plugin>
          <artifactId>maven-compiler-plugin</artifactId>
          <version>3.13.0</version>
        </plugin>
        <plugin>
          <artifactId>maven-surefire-plugin</artifactId>
          <version>3.3.0</version>
        </plugin>
        <plugin>
          <artifactId>maven-jar-plugin</artifactId>
          <version>3.4.2</version>
        </plugin>
        <plugin>
          <artifactId>maven-install-plugin</artifactId>
          <version>3.1.2</version>
        </plugin>
        <plugin>
          <artifactId>maven-deploy-plugin</artifactId>
          <version>3.1.2</version>
        </plugin>
        <!-- site lifecycle, see
https://maven.apache.org/ref/current/maven-
core/lifecycles.html#site_Lifecycle -->
        <plugin>
          <artifactId>maven-site-plugin</artifactId>
          <version>3.12.1</version>
        </plugin>
        <plugin>
          <artifactId>maven-project-info-reports-
plugin</artifactId>
          <version>3.6.1</version>
        </plugin>
      </plugins>
    </pluginManagement>
  </build>
</project>
```

File Name :

src/main/java/com/java/workspace/App.java

Code :

```
package com.java.workspace;

/**
 * Hello world!
 */
public class App {
    public static void main(String[] args) {
        System.out.println("Hello World!");
    }
}
```

File Name :

src/main/java/com/java/workspace/OrderManagement.java

Code :

```
package com.java.workspace;


import java.sql.*;


public class OrderManagement {
    public static void main(String[] args) {
        String url = "jdbc:mysql://localhost:3306/workspace";
// Change to your DB name
        String user = "root"; // Change to your DB username
        String password = "root"; // Change to your DB password


        try (Connection conn = DriverManager.getConnection(url,
user, password);
             Statement stmt = conn.createStatement()) {


            // 1. Create Table if not exists
            String createTableSQL = "CREATE TABLE IF NOT EXISTS
```

```
Orders (" +
                    "OrderID INT PRIMARY KEY AUTO_INCREMENT, "
+
                    "CustomerName VARCHAR(50), " +
                    "OrderDate DATE, " +
                    "Product VARCHAR(50), " +
                    "Quantity INT)";
            stmt.executeUpdate(createTableSQL);
            System.out.println("Table created successfully!");


            // 2. Insert Sample Data
            String insertSQL1 = "INSERT INTO Orders
(CustomerName, OrderDate, Product, Quantity) VALUES " +
                                "('Alice Smith', '2025-01-01',
'Laptop', 2)";
            String insertSQL2 = "INSERT INTO Orders
(CustomerName, OrderDate, Product, Quantity) VALUES " +
                                "('Bob Johnson', '2025-01-03',
'Phone', 1)";
            stmt.executeUpdate(insertSQL1);
            stmt.executeUpdate(insertSQL2);
            System.out.println("Data inserted successfully!");


            // 3. Fetch and display all records after insertion
            System.out.println("\nOrders After Insertion:");
            fetchAndDisplayOrders(stmt);


            // 4. Delete one record
            String deleteSQL = "DELETE FROM Orders WHERE
CustomerName = 'Alice Smith'";
            stmt.executeUpdate(deleteSQL);
            System.out.println("\nRecord deleted
successfully!");


            // 5. Fetch and display remaining records after
deletion
            System.out.println("\nRemaining Orders After
Deletion:");
            fetchAndDisplayOrders(stmt);
```

```
        } catch (SQLException e) {
            e.printStackTrace();
        }
    }


    // Utility function to fetch and display all orders
    private static void fetchAndDisplayOrders(Statement stmt)
throws SQLException {
        String fetchSQL = "SELECT * FROM Orders";
        try (ResultSet rs = stmt.executeQuery(fetchSQL)) {
            while (rs.next()) {
                System.out.println("OrderID: " +
rs.getInt("OrderID") + ", " +
                                   "CustomerName: " +
rs.getString("CustomerName") + ", " +
                                   "OrderDate: " +
rs.getDate("OrderDate") + ", " +
                                   "Product: " +
rs.getString("Product") + ", " +
                                   "Quantity: " +
rs.getInt("Quantity"));
            }
        }
    }
}
```

File Name :

src/test/java/com/java/workspace/AppTest.java

Code :

```
package com.java.workspace;

import static org.junit.jupiter.api.Assertions.assertTrue;
```